

```
=====
Spring JDBC / Spring DAO
=====
```

=> Spring JDBC is part of the Spring Framework

=> It is used to simplify DB operations in our java applications.

=> Spring JDBC handles resource management, exception handling, and reduces boilerplate code.

=> Spring JDBC provided predefined components and methods to simplify DB operations

Ex : JdbcTemplate, RowMapper, DataSource....

----- Normal JDBC Code-----

```
Connection conn = DriverManager.getConnection(url, user, pass);
PreparedStatement stmt = conn.prepareStatement("SELECT * FROM students");
ResultSet rs = stmt.executeQuery();
Student s = new Student();
while(rs.next()) {
    s.setId(rs.getInt("id"));
    s.setName(rs.getString("name"));
}
rs.close();
stmt.close();
conn.close();
```

----- Spring JDBC -----

```
String query = "SELECT * FROM students";
List<Student> students = jdbcTemplate.query(query, new StudentRowMapper());
```

```
=====
Key Components of Spring JDBC
=====
```

- 1) Data Source (I)
- 2) JdbcTemplate (c) (update & query)
- 3) RowMapper (I)

=> DataSource is an interface it is used to represent Database Connection Pooling.

=> JdbcTemplate is a core class of spring jdbc, which is providing methods to execute SQL queries.

=> RowMapper is used to convert resultset data to java objects.

@@@ Project Lombok Tutorial By Mr. Ashok : <https://youtu.be/8tDym-FxU0A?si=8M8PClnyApWpFZKV>

```
=====
Developing First Spring JDBC Application
=====
```

Step-1 : Setup MySQL DB Server and MySQL Workbench (DB Client)

@@ Reference Video : <https://www.youtube.com/watch?v=EsAIXPIsyQg>

Step-2 : Create Database & Database table

```
...  
show databases;  
create database sbms75;  
use sbms75;  
show tables;  
  
select * from student;  
  
CREATE TABLE student (  
    id INT PRIMARY KEY,  
    name VARCHAR(100),  
    city VARCHAR(100)  
);  
  
select * from student;  
...
```

Step-3 : Create Maven Project with required dependencies

- a) spring-context
- b) spring-jdbc
- c) mysql-driver

Step-4 : Create DTO class to represent data in object format.

```
public class Student {  
  
    private Integer id;  
    private String name;  
    private String city;  
  
    // setters & getters  
  
    // toString ( )  
  
}
```

Step-5 : Create DAO class to perform DB operations using JdbcTemplate class.

```
...  
@Repository  
public class StudentDao {  
  
    private JdbcTemplate jt;  
  
    public StudentDao(JdbcTemplate jt) {  
        this.jt = jt;  
    }  
  
    public int save(Student s) {  
        String sql = "INSERT INTO STUDENT (ID, NAME, CITY) VALUES (?, ?, ?)";  
        int rowsAffected = jt.update(sql, s.getId(), s.getName(), s.getCity());  
        return rowsAffected;  
    }  
}
```

Step-6 : Create AppConfig class and represent DataSource and JdbcTemplate as spring beans like below.

```
...  
  
@Configuration  
@ComponentScan(basePackages = "in.ashokit")  
public class AppConfig {
```

```

@Bean
public DataSource dataSource() {
    DriverManagerDataSource ds = new DriverManagerDataSource();
    ds.setDriverClassName("com.mysql.cj.jdbc.Driver");
    ds.setUrl("jdbc:mysql://localhost:3306/sbms75");
    ds.setUsername("root");
    ds.setPassword("root");
    return ds;
}

@Bean
public JdbcTemplate createJdbcTemplate(DataSource ds) {
    return new JdbcTemplate(ds); // CI
}

}
...

```

Step-7 :: Create main class to test the application

```

...
public class MainApp {

    public static void main(String[] args) {

        ApplicationContext ctxt = new AnnotationConfigApplicationContext(AppConfig.class);

        StudentDao dao = ctxt.getBean(StudentDao.class);

        Student s = new Student();
        s.setCity("Hyd");
        s.setName("Ashok");
        s.setId(101);

        int cnt = dao.save(s);

        System.out.println("Rows Effected :: " + cnt);

    }

}
...

```

=====

RowMapper class

=====

```

public class StudentMapper implements RowMapper<Student> {

    @Override
    public Student mapRow(ResultSet rs, int rowNum) throws SQLException {

        System.out.println("mapRow() called");

        Student s = new Student();

        s.setId(rs.getInt("id"));
        s.setName(rs.getString("name"));
        s.setCity(rs.getString("city"));

        return s;

    }

}

```

@Repository

```
public class StudentDao {  
    private JdbcTemplate jt;  
  
    public StudentDao(JdbcTemplate jt) {  
        this.jt = jt;  
    }  
  
    public List<Student> findAll() {  
        return jt.query("SELECT * FROM STUDENT", new StudentMapper());  
    }  
}
```

```
-----  
public class MainApp {  
    public static void main(String[] args) {  
        ApplicationContext ctxt = new AnnotationConfigApplicationContext(AppConfig.class);  
        StudentDao dao = ctxt.getBean(StudentDao.class);  
  
        List<Student> all = dao.findAll();  
        all.forEach(System.out::println);  
    }  
}
```